

ORIGINAL ARTICLE OPEN ACCESS

When Do Graph Neural Networks Help in Spatial Prediction?

Josiah Parry     | Renato Assunção  | Mark Janikas  | Marshall Mueller ¹Spatial Statistics Team, ESRI, California, USA |**Correspondence:** Josiah Parry (jparry@esri.com)**Received:** 30 August 2026 | **Revised:** ??? | **Accepted:** ???**Keywords:** spatial prediction | graph neural networks | spatial autocorrelation | spatial lag models | spatially structured covariates | spatial statistics

ABSTRACT

Graph neural networks are increasingly used for spatial prediction because message passing provides a flexible mechanism for incorporating neighborhood information. Yet the same ideas have long been central to spatial statistics through spatial lags, spatial error models, and spatially structured covariates. This paper asks when graph neural networks provide predictive gains beyond those classical mechanisms. We compare GraphSAGE with ordinary least squares, a GMM spatial error model, XGBoost, and XGBoost augmented with spatial lag features, across a diagnostic simulation design isolating nonlinear covariate effects, spatially autocorrelated covariates, spatially autocorrelated errors, and neighborhood spillovers, and across three empirical exercises in housing prices and election outcomes. All evaluation is inductive: held-out units are given their own graph, with no edges back to the training region. GNN advantages are not automatic. GraphSAGE clears every alternative only where the process contains a neighborhood term, and under inductive prediction residual spatial autocorrelation is no resource at all, with correctly specified least squares beating every flexible method and the spatial error model reducing to it. Explicit lag features are a strong and inconsistent competitor, recovering most of the graph-based gain in two of our applications and none of it in the third. Our most transferable finding concerns evaluation rather than models: random cross-validation builds the held-out graph at a fraction of the true density, so it does not flatter graph-based methods but handicaps them, halving a measured gain that blocked holdout recovers in full. We conclude that GNNs are best understood as learned spatial operators whose value depends on the spatial mechanism in the data and, just as much, on the design used to measure it.

1 | Introduction

Graph neural networks (GNNs) have rapidly become an important class of models for prediction on structured data (REFS). Their appeal in geographical applications is natural: spatial data are often observed on networks, lattices, irregular point patterns, administrative regions, or other units connected by adjacency, proximity, mobility, or interaction (REFS). Through message passing, GNNs provide a flexible mechanism for allowing predictions at one location to depend not only on local covariates, but also on information from neighboring locations. This makes them attractive for spatial prediction problems in which local context, spatial dependence, and neighborhood effects are expected to matter. (REFS)

At the same time, the central idea of using neighboring information is not new to spatial statistics. Spatial lags, spatial error models, spatial filtering, geographically weighted models, conditional

autoregressive models, and related approaches have long represented spatial dependence through carefully specified neighborhood structures (REFS). From this perspective, the recent success of GNNs raises a distinctly spatial-statistical question: are GNNs learning genuinely new forms of spatial structure, or are they flexible nonlinear versions of familiar spatial lag and spatial smoothing models?

This paper addresses that question through a controlled comparison of GraphSAGE, ordinary least squares, XGBoost, and spatially augmented versions of XGBoost. We focus on GraphSAGE because it is an inductive GNN architecture: after training, it can be applied to new graphs or new spatial regions, provided that comparable covariates and neighborhood relations are available. This inductive property is especially relevant for geographical prediction, where a model trained in one region may be used to predict

in another, or where predictions may be required for spatial units not observed during training.

Our starting point is the observation that GraphSAGE can be interpreted as a learned spatial lag model. Each message-passing layer aggregates information from neighboring units. Successive layers allow information to propagate across higher-order neighborhoods. Unlike classical spatial lag specifications, the aggregation and transformation of neighboring information are learned jointly with the prediction function. GraphSAGE therefore combines three ingredients: nonlinear covariate learning, graph-based neighborhood aggregation, and a form of adaptive spatial smoothing. The practical question is when this additional flexibility improves prediction enough to justify the greater modeling complexity.

We investigate this question using a simulation design that separates several mechanisms that are often confounded in empirical spatial prediction problems. These include nonlinear conditional mean functions, spatially autocorrelated covariates, spatially autocorrelated errors, neighborhood spillovers, and discontinuous spatial regimes. By isolating these mechanisms, the simulations allow us to ask whether GNN gains arise from nonlinear regression, from the use of spatially structured covariates, from residual spatial dependence, from genuine neighborhood effects, or from some combination of these sources.

A key feature of the comparison is the inclusion of spatially informed machine learning baselines. In addition to ordinary XGBoost, we consider XGBoost models augmented with spatial lag features, such as lagged covariates and higher-order neighborhood summaries. This benchmark is important because message passing in GNNs is closely related to the construction of spatially lagged predictors. Without such a comparison, any apparent advantage of a GNN could simply reflect the fact that it has access to neighborhood information unavailable to the competing models. We also emphasize prediction to unseen spatial regions. Random train-test splits can overstate predictive performance in spatial settings because nearby training observations may surround test observations (REFS). Spatial block holdout designs provide a more demanding evaluation: the model must predict in regions that were not used during training. This distinction is particularly important for GNNs, whose message-passing structure may perform very well when interpolating within a known spatial field but may behave differently when transferred to new regions.

The main contributions of this paper are as follows.

- **A conceptual bridge between GNNs and spatial statistics.** We interpret GraphSAGE as a flexible, nonlinear, learned spatial lag model and relate its message-passing mechanism to familiar spatial-statistical ideas such as spatial lags, neighborhood effects, and spatial smoothing.
- **A diagnostic simulation framework.** We introduce a set of controlled data-generating processes that isolate different spatial mechanisms, including nonlinear regression, spatially autocorrelated covariates, spatially autocorrelated errors, neighborhood spillovers, and spatial regimes.
- **A fair comparison with spatially informed baselines.** We compare GraphSAGE not only with ordinary least squares and standard XGBoost, but also with XGBoost models augmented by spatial lag features. This allows us to assess whether GNNs provide predictive value beyond explicit spatial feature engineering.

- **An evaluation based on spatial generalization.** We examine model performance under spatial holdout designs that require prediction in unseen regions, rather than relying only on randomly interspersed test observations.
- **Empirical illustrations.** We complement the simulations with applied examples involving housing price prediction and election prediction. These examples illustrate both the potential gains of GraphSAGE and the importance of comparing it with strong spatially informed alternatives.

The results suggest that the value of GNNs depends strongly on the underlying spatial mechanism. GNNs can substantially improve spatial prediction when the data-generating process contains nonlinear neighborhood effects, multiscale spatial context, or residual spatial structure that can be exploited through the graph. However, their advantage is not automatic. In several settings, much of the predictive gain can be recovered by simpler models using explicit spatial lag features. Thus, GNNs should not be viewed as replacements for spatial-statistical thinking, but as flexible learned spatial operators whose value depends on the spatial mechanism underlying the prediction problem.

2 | GNNs and spatial statistics

Related work section. Explain GraphSAGE as learned message passing. Relate it to WX, WX, spatial lags, spatial filtering, and neighborhood/context effects.

3 | Simulation design

The simulation study is designed to identify the conditions under which graph neural networks provide predictive gains in spatial regression. All simulations are conducted on a regular spatial lattice represented as a graph. Let $i = 1, \dots, n$ index spatial units, let s_i denote the location of unit i , and let W be a row-standardized spatial weights matrix constructed from the neighborhood graph. The response is denoted by Y_i , the local covariate vector by X_i , and the spatially autocorrelated covariates by $X_{sp,i}$. Across scenarios, we compare models under data-generating processes of the general form

$$Y_i = m(X_i, X_{sp,i}, s_i) + u_i,$$

where $m(\cdot)$ is the conditional mean function and u_i is either independent noise or a spatially autocorrelated residual. When spatially autocorrelated errors are present, we generate them as

$$u = (I - \rho W)^{-1} \varepsilon, \quad \varepsilon \sim N(0, \sigma^2 I),$$

where ρ controls the strength of residual spatial dependence. Spatially autocorrelated covariates are generated analogously as spatial error processes,

$$X_{sp} = (I - \lambda W)^{-1} \eta, \quad \eta \sim N(0, I),$$

with λ controlling the spatial autocorrelation of the covariate field.

The simulation design consists of five scenarios. Each scenario is intended to favor, or challenge, different modeling strategies. Ordinary least squares should perform well when the conditional

mean is linear and residuals are independent. XGBoost should perform well when the conditional mean is nonlinear but non-spatial. Spatially augmented XGBoost should perform well when predictive information is available through explicit spatial lag features. GraphSAGE should be most useful when prediction depends on neighborhood context in ways that are nonlinear, multiscale, or difficult to encode through fixed spatial features.

Every scenario is simulated on the same lattice under the same graph, so the only thing that changes between them is the data-generating equation. Table 1 gives the full specification.

The autocorrelation parameters are calibrated rather than assumed. We measured realized Moran's I across a grid of λ values on the KNN-15 graph and inverted that curve at each target, so the I_x and I_u labels on every result are measured quantities and not nominal settings.

Scenario A: Linear mean with independent errors. The baseline scenario is a standard linear regression model with independent errors:

$$Y_i = X_i^\top \beta + \varepsilon_i, \quad \varepsilon_i \stackrel{iid}{\sim} N(0, \sigma^2).$$

The covariates are independent across space and the residuals contain no spatial structure. This scenario is included as a negative control for graph-based methods. Since the data-generating process contains no neighborhood effect, spatial autocorrelation, or nonlinear structure, a graph neural network should not have a systematic advantage over a correctly specified linear model. If GraphSAGE improves substantially in this setting, the improvement would require explanation, since the graph contains no relevant predictive information beyond the local covariates.

Scenario B: Nonlinear mean with independent errors. The second scenario introduces nonlinear covariate effects while keeping the residuals independent:

$$Y_i = m(X_i) + \varepsilon_i, \quad \varepsilon_i \stackrel{iid}{\sim} N(0, \sigma^2).$$

The purpose of this scenario is to isolate nonlinear mean structure from residual spatial dependence. To evaluate whether spatial structure in the covariates affects model performance even when the errors are independent, we generate the covariates under three settings: no spatial autocorrelation, moderate spatial autocorrelation with $I_x = 0.4$, and stronger spatial autocorrelation with $I_x = 0.7$. Thus, the response may inherit spatial structure through the covariates, but there is no additional spatial dependence in the residual process. For example, we use nonlinear functions such as

$$m(X_i) = 2 \sin(X_{1i}) + X_{2i}^2 - 1.5X_{3i}X_{4i}.$$

We also consider threshold-type functions, such as

$$m(X_i) = \begin{cases} 2, & X_{1i} > 0, X_{2i} > 0, \\ -2, & X_{1i} < 0, X_{2i} > 0, \\ 0, & \text{otherwise.} \end{cases}$$

This scenario tests whether predictive gains are due simply to nonlinear regression. A flexible nonspatial learner such as XGBoost should perform well here, whereas ordinary least squares is misspecified. The comparison across the three covariate settings further distinguishes nonlinear prediction from the exploitation

of spatially structured covariates. Since the residuals are independent, any substantial advantage of GraphSAGE over XGBoost would suggest that the GNN is benefiting from neighborhood aggregation of spatially structured covariates or from model flexibility, rather than from residual spatial autocorrelation.

Scenario C: Linear mean with spatially autocorrelated errors. The third scenario corresponds to a classical spatial error setting:

$$Y_i = X_i^\top \beta + u_i, \quad u = (I - \rho W)^{-1} \varepsilon.$$

Here the conditional mean is linear, but the residual field is spatially autocorrelated. As in Scenario B, we generate the covariates under three settings: no spatial autocorrelation, $I_x = 0.4$, and $I_x = 0.7$. This allows us to separate the effect of spatially structured covariates from the effect of spatially structured residuals. The strength of residual spatial autocorrelation is controlled separately through the spatial error process for u .

This scenario is important because it creates a setting in which classical spatial statistical models are well matched to the data-generating process. The conditional mean is linear, so ordinary least squares is correctly specified for $E(Y_i | X_i)$, but it ignores the spatial dependence in the residual field. A spatial error model, or a spatial filtering alternative, should therefore provide a strong classical benchmark.

This scenario distinguishes two sources of predictive performance. A model may predict well because it estimates the conditional mean accurately, because it exploits spatial structure in the covariates, or because it interpolates a spatially smooth residual field. Under random holdout, test units may be surrounded by nearby training units, making residual interpolation relatively easy. Under spatial block holdout, entire regions are withheld, making prediction more demanding. We therefore evaluate this scenario under both random and spatially blocked test designs.

Scenario D: Nonlinear mean with spatially autocorrelated errors. The fourth scenario combines nonlinear covariate effects with residual spatial dependence:

$$Y_i = m(X_i) + u_i, \quad u = (I - \rho W)^{-1} \varepsilon.$$

As in the previous scenarios, the covariates are generated under three levels of spatial structure: no spatial autocorrelation, $I_x = 0.4$, and $I_x = 0.7$. This creates versions of the same nonlinear regression problem in which spatial structure may enter through the covariates, through the residual field, or through both mechanisms simultaneously.

A representative conditional mean is

$$m(X_i) = 1.5X_{1i} + 2 \sin(X_{2i}) + X_{3i}^2 + 1.5X_{4i}^2.$$

When one of the covariates is spatially autocorrelated, this specification also induces spatial structure in the conditional mean. The residual process u then adds an independent source of spatial dependence not explained by the covariates.

This scenario is deliberately unfavorable to any single simple model class. Ordinary least squares is misspecified because the mean is nonlinear. A linear spatial error model captures residual spatial dependence but not the nonlinear mean. Ordinary XGBoost captures nonlinear covariate effects but does not explicitly model residual spatial autocorrelation. Spatially augmented XGBoost may improve by using explicit neighborhood features,

TABLE 1 | Simulation parameters. All scenarios share the lattice, the graph, and the fold structure.

Parameter	Value
Units n	7,000
Lattice	100 × 70 regular grid
Graph W	k -nearest neighbors, $k = 15$, row standardized
Folds	6, inductive
Train / validation / test per fold	5,251 / 583 / 1,166
β	(1, 0.3, 0.3, 0.3, 0.3, 0.3)
σ	1
λ for $I_x = 0.4/0.7$	0.857/0.961
ρ for $I_u = 0.4/0.8$	0.857/0.975
Spillover share of $\text{Var}(Y)$, Scenario E	0.2
GraphSAGE hidden dimensions	56, 32, 16
Epochs / learning rate / patience	500/0.01/20
XGBoost trees	500

while GraphSAGE may benefit by combining nonlinear feature learning with graph-based neighborhood aggregation.

This scenario is therefore central to the simulation study. It represents a realistic case in which spatial prediction requires flexible modeling of the conditional mean, recognition of spatially structured covariates, and some ability to exploit residual spatial structure. The comparison between GraphSAGE and spatially augmented XGBoost is especially informative here: if their performance is similar, much of the GNN advantage can be explained by nonlinear regression plus spatial feature engineering; if GraphSAGE performs better, this suggests that learned message passing provides value beyond fixed spatial lag features.

Scenario E: Neighborhood covariate effects. The fifth scenario introduces explicit neighborhood effects. The response depends not only on local covariates but also on covariates observed at neighboring locations:

$$Y_i = X_i^\top \beta + \theta(WX_1)_i + \varepsilon_i.$$

This corresponds to spatial spillovers, diffusion, accessibility effects, local exposure, or other contextual processes in which neighboring covariates directly influence the response.

Scenario E is the only scenario whose data-generating process contains a lag term. In A and C the mean is linear in a unit's own X ; in B and D it is nonlinear but still a function of X_i alone. In all four, $(WX)_i$ is noise by construction. A finding that spatial lag features add nothing in those scenarios restates how the equations were written and is not evidence about lag features. Only Scenario E can speak to that question, which makes its parameterization consequential.

We set θ from a target share of response variance rather than fixing it. With row-standardized KNN-15 weights and an independent X_1 , $\text{Var}(WX_1) = 1/k \approx 0.067$, so a moderate-looking θ places the spillover at roughly one percent of $\text{Var}(Y)$, below what any method could detect. Solving

$$s = \frac{\theta^2 \text{Var}(WX_1)}{\text{Var}(X^\top \beta) + \theta^2 \text{Var}(WX_1) + \sigma^2}$$

for θ at $s = 0.2$ makes the spillover large enough to find and keeps the two Scenario E settings comparable when the covariate field's own autocorrelation changes $\text{Var}(WX_1)$.

We run Scenario E at two levels of covariate autocorrelation, and the contrast between them is the point. At $I_x = 0$, WX_1 is nearly orthogonal to X_1 : the neighborhood effect is information a unit

cannot obtain from itself, and recovering it requires the graph. At $I_x = 0.4$, WX_1 correlates with X_1 and part of the effect is reachable from a unit's own covariate without consulting the graph at all. Real covariate surfaces are usually in the second position.

This is the scenario in which a graph neural network has the clearest structural justification, since message passing is designed precisely to aggregate information from neighboring units. It is also where the comparison with spatially augmented XGBoost matters most, because $(WX)_i$ is exactly the quantity in the data-generating equation. If explicit lag features recover the effect, the GNN's architectural advantage is small.

A scenario we removed: discontinuous spatial regimes. An earlier version of this design included a sixth scenario with spatial regimes. The study region is divided into two or more contiguous subregions, and each region follows a different regression function:

$$Y_i = \begin{cases} X_i^\top \beta^{(1)} + \varepsilon_i, & s_i \in A_1, \\ X_i^\top \beta^{(2)} + \varepsilon_i, & s_i \in A_2. \end{cases}$$

A nonlinear version is

$$Y_i = \begin{cases} m_1(X_i) + \varepsilon_i, & s_i \in A_1, \\ m_2(X_i) + \varepsilon_i, & s_i \in A_2. \end{cases}$$

We ran this scenario and then removed it, because it cannot answer the question it was built to ask.

The regime was identifiable only from position. Covariates were drawn from the same distribution in both halves, and no method in the comparison observes coordinates. Neither a unit's own covariates nor any aggregation of its neighbors' covariates carries information about which regime it occupies. Every method scored an out-of-sample R^2 near 0.01, and the scenario separated nothing.

This is worth stating rather than omitting. A regime shift invisible in the features is invisible to every method, graph-based or not, and a null result there says nothing about oversmoothing. Testing oversmoothing requires a regime that is at least partially identifiable, which in practice means a shift in the covariate distribution itself. The King County to Ames transfer in Section 6 is that case, on real data: two housing markets whose covariate distributions genuinely differ.

Summary of scenarios. Table 2 summarizes the five scenarios and the spatial mechanism isolated by each one.

TABLE 2 | Simulation scenarios used to diagnose when graph neural networks improve spatial prediction.

Scenario	Mean structure	Error structure	Purpose
A	Linear $X\beta$	iid	Baseline negative control
B	Nonlinear $m(X)$	iid	Tests nonlinear learners
C	Linear $X\beta$	Spatial error	Classical spatial statistics case
D	Nonlinear $m(X, X_{sp})$	Spatial error	Nonlinear mean with residual spatial dependence
E	Neighborhood effect $\theta W X_1$	iid	Tests graph-based neighborhood aggregation

Across scenarios, we vary the strength of spatial structure in the covariates and residuals. Let I_x denote the Moran's I of the spatially autocorrelated covariate field and I_u denote the Moran's I of the residual field. We consider a small number of interpretable levels, for example

$$I_x \in \{0, 0.4, 0.7\}, \quad I_u \in \{0, 0.4, 0.8\}.$$

The purpose is not to exhaust all possible combinations, but to create a compact design that distinguishes the mechanisms through which spatial structure may affect prediction.

The resulting simulation design is therefore diagnostic rather than merely comparative. A model that performs well in Scenario B is primarily capturing nonlinear covariate effects. A model that improves in Scenario C is exploiting residual spatial autocorrelation. A model that performs well in Scenario D is combining flexible nonlinear prediction with the ability to exploit residual spatial dependence. A model that performs well in Scenario E is using neighborhood information. This organization makes it possible to interpret differences among OLS, XGBoost, spatially augmented XGBoost, classical spatial models, and GraphSAGE in terms of the spatial mechanisms present in the data-generating process.

4 | Competing methods

We compare GraphSAGE with a set of increasingly spatially informed baselines. The goal is not merely to compare a graph neural network with standard nonspatial methods, but to determine whether its predictive gains remain after simpler spatial-statistical and spatially engineered alternatives are considered. The competing methods therefore include ordinary least squares, XGBoost, XGBoost augmented with spatial lag features, and classical spatial models for scenarios in which the data-generating process contains residual spatial dependence.

Ordinary least squares. The first baseline is ordinary least squares (OLS). For each training sample, we fit the linear model $Y_i = X_i^\top \beta + \varepsilon_i$, using only the local covariates observed at unit i . OLS is included as a reference model because it is correctly specified in the simplest linear iid scenario and provides a useful benchmark for assessing the effect of nonlinearity and spatial dependence.

OLS is expected to perform well in Scenario A, where the conditional mean is linear and the errors are independent. Its performance should deteriorate when the conditional mean is nonlinear, when residuals are spatially autocorrelated, or when the response depends on neighboring covariates. Thus, OLS serves as a baseline for identifying how much predictive improvement is obtained by adding nonlinear modeling capacity, spatial structure, or both.

XGBoost. The second baseline is XGBoost (REF), a flexible tree-based machine learning method. We fit XGBoost using the same local covariates X_i available to OLS, but without explicit spatial lag features. The model can therefore capture nonlinearities, interactions, and threshold effects in $m(X_i)$, the conditional mean in the model $Y_i = m(X_i) + \varepsilon_i$. This makes XGBoost a strong competitor in scenarios where the main source of difficulty is nonlinear covariate response rather than spatial dependence.

XGBoost is expected to perform well in scenarios with nonlinear but nonspatial data-generating processes. However, because the baseline version uses only local covariates, it does not explicitly represent residual spatial autocorrelation or neighborhood spillovers. Its performance therefore helps distinguish gains due to nonlinear function approximation from gains due to spatial information.

XGBoost with spatial lag features. To provide a stronger spatially informed machine learning baseline, we also consider XGBoost augmented with spatial lag features. Let W denote the row-standardized spatial weights matrix. For each spatial unit, the augmented predictor vector pairs every covariate with its neighborhood mean: X_i and $(WX)_i$. Every other method in the comparison is fit on X_i alone, so this arm isolates the value of supplying neighborhood covariate information explicitly.

This model is important because message passing in a GNN is closely related to the construction of spatial lag features. If GraphSAGE outperforms ordinary XGBoost but not XGBoost with spatial lags, then much of the GNN advantage may be attributable to access to neighborhood covariates rather than to a fundamentally different modeling capability. Conversely, if GraphSAGE outperforms spatially augmented XGBoost, this suggests that learned nonlinear aggregation provides predictive value beyond fixed spatial feature engineering.

For the spatially augmented XGBoost model, spatial lag features are constructed using only the graph available within the relevant training or prediction setting. In spatial holdout experiments, care is taken to avoid using response information from the held-out region. This distinction is essential because spatial lag construction can otherwise introduce leakage between training and test observations.

Classical spatial models. For scenarios with spatially autocorrelated errors, we include a classical spatial model as an additional baseline. The natural benchmark is the spatial error model, $Y = X\beta + u$, with $u = \lambda Wu + \varepsilon$, or equivalently, $u = (I - \lambda W)^{-1} \varepsilon$. This model is appropriate when the conditional mean is linear but the residual field is spatially autocorrelated. It provides a direct classical counterpart to Scenario C, where the data-generating process is designed precisely as a linear mean with spatially structured errors.

When computational or implementation constraints make full spatial error estimation difficult at the scale of the simulations, a

spatial filtering or residual-smoothing alternative can be used instead. For example, one may augment the regression with spatial eigenvector filters, Moran eigenvectors, or other low-dimensional spatial basis functions. Another practical alternative is a two-stage procedure in which a first-stage model is fit to the covariates and the residuals are then spatially smoothed. The essential requirement is that the comparison include at least one method designed explicitly to handle residual spatial autocorrelation.

The purpose of this baseline is not to claim that a single spatial error model exhausts the classical spatial statistics literature. Rather, it ensures that GraphSAGE is compared with a model that is structurally well matched to spatially autocorrelated residuals. Without such a benchmark, any GNN advantage in spatial error scenarios could simply reflect the absence of an appropriate classical spatial competitor.

GraphSAGE. The graph neural network considered in this paper is GraphSAGE. GraphSAGE is an inductive message-passing architecture: it learns a function for aggregating information from a node's neighborhood and can then apply that function to new nodes or new graphs. This property is particularly relevant for spatial prediction, where the goal is often to predict in spatial regions not used during training.

At each layer, GraphSAGE updates the representation of unit i by combining its current representation with an aggregation of the representations of its neighbors. In simplified form, a layer can be written as

$$h_i^{(\ell+1)} = \sigma \left(A^{(\ell)} h_i^{(\ell)} + B^{(\ell)} \text{AGG}\{h_j^{(\ell)} : j \in N(i)\} \right),$$

where $h_i^{(\ell)}$ is the representation of unit i at layer ℓ , $N(i)$ is the set of neighbors of i , $\text{AGG}(\cdot)$ is a neighborhood aggregation function, $\sigma(\cdot)$ is a nonlinear activation function, and $A^{(\ell)}$ and $B^{(\ell)}$ are learned parameters. The initial representation $h_i^{(0)}$ is given by the covariate vector X_i , possibly augmented with additional features. This formulation makes clear the connection between GraphSAGE and spatial lag models. A single message-passing layer uses information similar to a first-order spatial lag. Multiple layers allow information to propagate across higher-order neighborhoods, analogous to using WX , W^2X , and higher-order lag features. The difference is that GraphSAGE learns nonlinear transformations and aggregation weights jointly with the prediction task, rather than requiring these spatial features to be specified in advance. GraphSAGE is expected to be most useful when the data-generating process contains neighborhood effects, nonlinear spatial context, or residual spatial structure that can be exploited through the graph. However, its advantage is not automatic. In scenarios where the true process is linear and nonspatial, or where spatial structure can be adequately represented by explicit lag features, simpler models may perform similarly while being easier to estimate and interpret.

Comparison strategy. The methods are compared across the simulation scenarios described in Section 3. The comparison is diagnostic. OLS establishes performance under a simple linear specification. XGBoost evaluates the benefit of nonlinear nonspatial learning. XGBoost with spatial lags evaluates the benefit of explicit spatial feature engineering. Classical spatial models evaluate the benefit of modeling residual spatial autocorrelation. GraphSAGE evaluates whether learned message passing provides additional predictive value beyond these alternatives. Taken together, these methods form a hierarchy of increasingly

spatial and flexible predictors: from linear nonspatial regression, to nonlinear machine learning, to explicit spatial feature engineering, to classical spatial error modeling, and finally to learned graph-based aggregation. This hierarchy is designed to determine whether GraphSAGE provides predictive value beyond simpler and more interpretable spatial alternatives.

The held-out design. Every experiment is inductive. Held-out units are removed from the lattice and given their own graph, built by running the same KNN-15 construction over the held-out units alone. No edge crosses the boundary. A model therefore never sees a training unit's covariates or response when predicting, and spatial lag features for held-out units are computed on the held-out graph rather than the full one. This closes the leakage path that a naive lag construction would open.

We use two holdout designs, each with six folds, on identical data.

- **Spatial block.** The lattice is cut into six contiguous vertical strips, each spanning its full height. Each strip is held out in turn. A held-out unit's neighbors are almost all held out with it, so the held-out graph closely reproduces the graph that generated the data.
- **Random.** Six folds of scattered units. A held-out unit's true neighbors are mostly training units, so its held-out graph is built at one sixth the lattice density and its neighborhoods span roughly 2.4 times the radius of the graph the data were generated on.

This difference is not incidental, and Section 5 shows it drives one of the study's main results. Random holdout is the conventional choice, but under an inductive protocol it does not merely make prediction easier. It changes the graph the model is tested on.

5 | Simulation results

We now compare the predictive performance of OLS, XGBoost, spatially augmented XGBoost, classical spatial models, and GraphSAGE across the six simulation scenarios. The purpose of the simulation study is not only to identify the best-performing model, but to determine which spatial mechanisms are responsible for performance differences. We therefore organize the results by scenario and interpret each comparison in relation to the data-generating process.

All results are out-of-sample R^2 , averaged over six folds, under the inductive holdout described in Section 4. We report the spatial block design throughout unless the random design is named explicitly. Two versions of GraphSAGE are fit in every fold, with and without LayerNorm, seeded identically so that the normalization is the only difference between them. The spatial error model is fit only in Scenarios C and D, the two with spatially autocorrelated errors; elsewhere it has nothing to estimate that OLS does not. Rather than focusing on average rankings, we emphasize how the relative ordering changes as the data-generating process moves from nonspatial linear regression to nonlinear regression, spatially autocorrelated residuals, and neighborhood spillovers.

5.1 | Scenario A: Linear mean with independent errors

Scenario A is the baseline case in which the conditional mean is linear and the errors are independent. OLS reaches $R^2 = 0.359$, GraphSAGE 0.348 and 0.350 with LayerNorm, XGBoost 0.226 and 0.238 with lag features.

The negative control holds. GraphSAGE lands within 0.011 of a correctly specified linear model and does not beat it. When the response contains no spatial dependence, no nonlinear covariate effects and no neighborhood spillovers, the graph carries nothing beyond the local covariates, and message passing finds nothing to exploit. Gains in later scenarios are therefore not a general property of the architecture.

XGBoost is the method that suffers here, 0.13 below OLS. Tree ensembles approximate a linear surface with piecewise-constant splits and pay for it. This is worth noting because it recurs: XGBoost’s deficit in Scenarios A and C is a statement about linearity, not about space.

5.2 | Scenario B: Nonlinear mean with independent errors

Scenario B introduces nonlinear covariate effects while preserving independent errors. The covariates are generated under three levels of spatial structure: spatially independent covariates, moderately spatially autocorrelated covariates with $I_x = 0.4$, and strongly spatially autocorrelated covariates with $I_x = 0.7$. Thus, this scenario allows us to assess whether spatially structured covariates alone create an advantage for graph-based prediction when there is no residual spatial dependence and no neighborhood spillover in the data-generating process.

The ordering is the same at all three covariate settings. At $I_x = 0$, GraphSAGE reaches 0.968, XGBoost 0.957, and OLS 0.779. At $I_x = 0.7$ the corresponding values are 0.954, 0.940 and 0.756. Nonlinear learning is what matters, and it is worth roughly 0.18 of R^2 over OLS.

Spatial lag features add nothing: 0.955 against 0.957 at $I_x = 0$, and identical to three digits at $I_x = 0.7$. This is a property of the equations rather than a result, since Y is a function of X_i alone and $(WX)_i$ is noise by construction. We note it because it sets a scale. Across Scenarios A through D, where the lag features are provably uninformative, the largest movement they produce is 0.023. Differences below roughly 0.02 in this design are not readable.

Moving from independent covariates to $I_x = 0.7$ does not change the relative ordering. Spatial autocorrelation in the covariates, on its own, creates no GNN advantage when the response depends only on local covariate values and the residuals are independent. This pattern suggests that good performance in Scenario B should be interpreted primarily as evidence of flexible nonlinear mean estimation, not as evidence of spatial learning. The spatial structure in X is already observed by all methods through the covariates. Since the data-generating process does not include residual spatial autocorrelation or neighborhood effects, message passing has little additional spatial information to exploit. Therefore, the similarity between XGBoost and GraphSAGE in this scenario supports the interpretation that GNN gains in later scenarios arise from genuinely spatial mechanisms rather than from the mere presence of spatially autocorrelated covariates.

5.3 | Scenario C: Linear mean with spatially autocorrelated errors

Scenario C keeps the conditional mean linear but introduces spatially autocorrelated errors. The covariates are generated under different levels of spatial structure, but the key source of additional predictive difficulty is the residual process:

$$Y_i = X_i^\top \beta + u_i, \quad u = (I - \rho W)^{-1} \varepsilon.$$

Thus, the model can perform well either by estimating the linear conditional mean or by exploiting the spatial structure in the residual field.

The classical models win, and their margin grows with the autocorrelation. At $I_u = 0$, OLS and the spatial error model both reach 0.355, GraphSAGE 0.342. At $I_u = 0.8$ the classical models reach 0.377 and 0.378 while GraphSAGE falls to 0.218. The gap widens from 0.013 to 0.159 as the residual field becomes more structured.

This is the opposite of what a reader might expect, and the inductive protocol is the reason. The held-out region’s residual field is never observed. No edge crosses the boundary, so there is no path by which residual information can reach a held-out unit. Because the errors in Scenario C are independent of X by construction, the best any method can do is estimate $E(Y | X)$ accurately, and OLS is correctly specified for exactly that. Residual spatial autocorrelation is not a resource here. It is noise that a flexible method can overfit.

The spatial error model matches OLS to three digits in every setting. This is not a failure of the model but a consequence of the protocol: under inductive prediction its forecast reduces to $X_{\text{test}}^\top \hat{\beta}$, since the term that distinguishes it from OLS is unobservable outside the training region. Where the held-out region is genuinely disjoint, a spatial error model offers no predictive advantage over least squares, whatever its advantages for inference.

GraphSAGE’s deterioration is the informative part. Its flexibility lets it fit structure in the training residual field that does not transfer, and the more structured that field, the more there is to overfit. LayerNorm mitigates this substantially, raising 0.218 to 0.307 at $I_u = 0.8$, its largest effect anywhere in the simulations. Neither version reaches OLS.

Spatial lag features do not help either, moving XGBoost from 0.222 to 0.241 at $I_u = 0.8$, within the noise established in Scenario B. The spatial signal here lies in u , not in omitted transformations of X , and neighborhood summaries of the covariates cannot recover it.

5.4 | Scenario D: Nonlinear mean with spatially autocorrelated errors

Scenario D combines nonlinear covariate effects with spatially autocorrelated residuals:

$$Y_i = m(X_i) + u_i, \quad u = (I - \rho W)^{-1} \varepsilon.$$

The covariates are again considered under different spatial correlation settings. Thus, spatial structure may enter the problem through the covariates, through the residual field, or through both. This scenario is the most realistic of the simulation settings

because it combines flexible mean structure with residual spatial dependence.

Nonlinearity dominates. GraphSAGE reaches 0.938, XGBoost 0.913, and OLS collapses to 0.456. The spatial error model tracks OLS at 0.457, for the same reason it did in Scenario C. Adding residual spatial autocorrelation on top barely moves any method: across the three settings, from $I_u = 0$ to $I_u = 0.8$, GraphSAGE stays between 0.936 and 0.938.

Two things follow. First, this scenario is much easier than Scenario C despite containing everything Scenario C contains, because a strong nonlinear mean is recoverable from the covariates while a residual field beyond the training boundary is not. Second, the classical models' failure here is misspecification of the mean, not a spatial failure.

Spatial lag features change nothing: 0.914 against 0.913. We flag this because the natural reading of the gap between GraphSAGE and XGBoost is that the GNN's neighborhood aggregation is doing the work, and that reading is wrong. There is no neighborhood term in this data-generating process. $(WX)_i$ is noise here exactly as in Scenario B, and the 0.025 that separates GraphSAGE from XGBoost is a difference between two nonlinear function approximators, not evidence of spatial learning.

Scenario D therefore does not support attributing GNN gains to message passing. For that, the data-generating process has to contain a neighborhood term, which is what Scenario E supplies.

5.5 | Scenario E: Neighborhood covariate effects

Scenario E is the only scenario whose data-generating process contains a neighborhood term, and it is therefore the only one that can speak to whether graph-based aggregation or explicit lag features recover neighborhood structure.

It is also a scenario with a known ceiling. Because $\sigma^2 = 1$ against $\text{Var}(Y) \approx 1.9$, roughly half of the response is irreducible noise, and no method can exceed $R^2 \approx 0.48$. The linear signal alone is worth $\text{Var}(X^\top \beta) / \text{Var}(Y) \approx 0.28$. The spillover is the remaining 0.20, and that is what the comparison is for.

Under spatial block holdout, the graph-based methods recover the spillover. At $I_x = 0$, GraphSAGE reaches 0.433, about ninety percent of the available 0.48. Lag features recover a large part of it, 0.334 against XGBoost's 0.163, but GraphSAGE clears them by 0.099. OLS, which cannot see the graph at all, attains 0.285, essentially its own ceiling of 0.28.

Under random holdout, the same methods recover almost nothing. GraphSAGE falls from 0.433 to 0.240, and lag features from 0.334 to 0.166, a gain of 0.009 over plain XGBoost that sits inside the noise floor of Scenario B. OLS is unchanged at 0.285, and plain XGBoost is unchanged at 0.157. The methods that consult the graph lose roughly half their performance; the methods that ignore it are untouched.

The mechanism is the held-out graph, not the difficulty of the region. A random fold holds out one sixth of the lattice, so the held-out graph is built at one sixth the density and its KNN-15 neighborhoods span about 2.4 times the radius of the W that generated Y . The lag operator at prediction time is a different operator from the one in the data-generating equation. On an independent covariate field a coarser average is nearly uncorrelated with the finer one that drives Y , so the signal is unrecoverable. Under block holdout the strips preserve local density, the held-out graph closely reproduces W , and the signal survives.

The $I_x = 0.4$ column confirms this reading. There the covariate field is smooth, so a coarser neighborhood average still correlates with the finer one, and the random-holdout penalty shrinks: GraphSAGE falls only from 0.510 to 0.448, and lag features retain a real gain. The degradation is worst precisely where the graph is the only route to the signal.

This has a practical consequence that we think is the most transferable result in the simulation study. Random cross-validation is the conventional default, and under an inductive graph protocol it is not merely optimistic. It systematically understates graph-based methods, because it tests them on a graph the process never ran on. This inverts the usual expectation that random splits flatter spatial models and blocked splits penalize them.

5.6 | Summary of simulation findings

Four findings survive the simulations.

GraphSAGE wins only where the process contains a neighborhood term. It matches OLS in Scenario A, beats XGBoost by a margin attributable to function approximation in B and D, loses to OLS in C, and clears every alternative in E. Its advantage is not a property of the architecture but of the correspondence between message passing and a spillover in the data-generating equation.

Under inductive prediction, residual spatial autocorrelation is not a resource. Scenario C makes this sharp: the classical models win, their margin grows with I_u , and the spatial error model reduces to OLS because the term distinguishing it is unobservable beyond the training boundary. A flexible method has more to overfit as the residual field becomes more structured, and GraphSAGE does.

Explicit lag features recover most but not all of a neighborhood effect, when they are measured on the right graph. In Scenario E under block holdout they close about three quarters of the distance between XGBoost and GraphSAGE. The remaining 0.099 is what learned aggregation buys over a fixed first-order lag. **The holdout design determines whether any of this is visible.** Scenario E scores 0.433 under blocked holdout and 0.240 under random holdout for the same method on the same data. The difference is not regional difficulty but the graph the method is tested on. Any evaluation of a graph-based spatial model has to state how the held-out graph was constructed, because that construction, not the model, can dominate the result.

These results support viewing GraphSAGE as a learned spatial operator rather than a general replacement for spatial-statistical models. Where the spatial mechanism is linear, or lies in the residuals, or is absent, simpler and more interpretable models match or beat it.

6 | Empirical illustrations

The simulations isolate mechanisms but cannot settle magnitudes, because their conditional means are known functions of each unit's own covariates. The empirical exercises carry that argument.

We consider three, in increasing order of what they ask of a model. The first predicts housing prices within King County, Washington, where held-out units come from the same market. The second predicts county-level election outcomes with an entire state held out, so the target region has its own graph. The

TABLE 3 | Scenario E, out-of-sample R^2 under two holdout designs. Only the methods that consult the graph respond to the design.

Method	$I_x = 0$		$I_x = 0.4$	
	Block	Random	Block	Random
OLS	0.285	0.285	0.422	0.429
XGBoost	0.163	0.157	0.291	0.296
XGBoost + lags	0.334	0.166	0.407	0.331
GraphSAGE	0.433	0.240	0.510	0.448
GraphSAGE + LayerNorm	0.434	0.240	0.511	0.452

third trains on King County and predicts Ames, Iowa, where the target is a different market with a different covariate distribution. Each uses the same roster and the same inductive protocol as the simulations. Held-out units are always given their own graph, and spatial lag features for held-out units are always computed on it.

6.1 | King County housing prices

The data are 21,613 King County sales from 2014–2015, with eight structural covariates. The response is log price, centered within the region. We use four-fold cross-validation, with each held-out quarter given its own KNN-30 graph.

GraphSAGE with LayerNorm reaches $R^2 = 0.824$ and plain GraphSAGE 0.815, against 0.759 for XGBoost with spatial lags, 0.592 for XGBoost, and 0.530 for OLS. The ordering on MAE is the same: 0.164, 0.169, 0.198, 0.262, 0.290.

Two things are worth separating. Spatial lag features are worth 0.167 of R^2 over plain XGBoost, far more than anywhere in the simulations, which is what we would expect when price genuinely depends on location. Learned aggregation is worth a further 0.065 over those features. Neighborhood information is doing most of the work, and explicit lags capture most of it.

This is prediction into unseen parts of a market the model was fit on. It is the easiest of the three exercises, and every method's performance here should be read as an upper bound on what it achieves in the two that follow.

6.2 | Election prediction with held-out states

The data are 3,108 counties with fourteen demographic and economic covariates; the response is county-level Democratic two-party vote share. Each state is held out in turn, trained on all counties outside it, and predicted using a queen-contiguity graph built over the held-out state alone. We report the 39 states with at least 20 counties, where the held-out graph has enough structure for an R^2 to be stable.

The graph-based methods win, and consistently. GraphSAGE beats XGBoost with spatial lags on 32 of 39 states by R^2 and 31 of 39 by MAE. Averaged over states this is 0.749 against 0.681. One of the two GraphSAGE variants is the best method in 26 of the 39 states.

Spatial lag features add nothing here. XGBoost with lags improves on plain XGBoost in 22 of 39 states, and its mean R^2 is 0.003 lower. This is a coin flip, and it stands in flat contradiction to the King County result, where the same features were worth 0.167. The per-state swings are large in both directions, from +0.151 in

New York to -0.167 in Idaho, so the null is not an absence of effect but a cancellation of effects.

We do not have a confirmed explanation. The natural candidate is that a held-out state's contiguity graph loses every out-of-state neighbor, so boundary counties carry lag features measured on a different operator than the training counties. But only boundary counties are affected, and this is far milder than the density change that drives the Scenario E result. We report the discrepancy rather than resolve it.

The classical models fail in a specific way. OLS and SEM are competitive on R^2 for many states but miss the level by up to 29 and 27 percentage points on the worst one, against 8 to 11 for the graph-based methods. The spatial error model is again close to OLS, for the reason Scenario C established. A method can rank counties within a state correctly and still be unusable if it places the whole state in the wrong place.

Accuracy and residual structure come apart. XGBoost leaves the least spatially clustered residuals of any method, $I = 0.178$ against 0.353 for GraphSAGE, while predicting less accurately. Low residual autocorrelation is often read as evidence that a model has absorbed the spatial structure. Here it is not, and it should not be used on its own as a model-selection criterion.

6.3 | Transfer from King County to Ames

The model is fit on all 21,613 King County sales and used to predict 2,930 Ames sales from 2006–2010. Both responses are log price centered within their own region, so the transfer is asked to reproduce relative variation rather than the price level, which is not identified across markets. A KNN-30 graph is built over Ames alone. This exercise adds distribution shift to spatial transfer: the two markets differ in period, price level, and housing stock.

The spatial error model is omitted here. King County's covariates satisfy `sfft_living = sfft_above + sfft_basement` exactly, leaving the design matrix rank deficient, which OLS tolerates by dropping a coefficient but GMM estimation does not.

This is the one exercise where LayerNorm decides the outcome. Plain GraphSAGE is the second-best model inside King County at 0.815 and falls to 0.424 on Ames. With LayerNorm it goes from 0.824 to 0.605. The two models differ in one argument, are seeded identically, and see identical data.

The calibration slope shows what fails. Plain GraphSAGE reaches 0.282 in Ames, meaning its predictions are roughly three and a half times more dispersed than the values they are predicting, with a bias of -0.233 . Ames' covariate distribution differs from King County's, and in an unnormalized network that shift propagates through the layers and is amplified. LayerNorm recenters each node's hidden vector using only that node's own values, so

TABLE 4 | Held-out states, averaged over 39 states. Bias is truth minus prediction, in percentage points; residual I is Moran's I of the held-out residuals on the state's own graph.

Method	MAE	R^2	mean bias	max bias	residual I
GraphSAGE	5.56	0.749	2.94	8.05	0.353
GraphSAGE + LayerNorm	6.44	0.751	3.23	10.77	0.328
XGBoost	6.38	0.684	3.05	9.85	0.178
XGBoost + lags	6.38	0.681	3.10	9.92	0.270
OLS	7.95	0.630	5.07	29.22	0.353
SEM	8.33	0.667	6.31	26.56	0.370

TABLE 5 | King County to Ames. R^2 within King County is four-fold cross-validation; Ames columns are transfer from a model fit on all of King County. The calibration slope is the coefficient from regressing observed on predicted values; one is correct, below one indicates predictions more dispersed than the truth, above one indicates predictions compressed toward their mean.

Method	R^2 King County	R^2 Ames	bias	calibration slope
GraphSAGE + LayerNorm	0.824	0.605	−0.044	1.575
XGBoost + lags	0.759	0.548	+0.050	0.910
GraphSAGE	0.815	0.424	−0.233	0.282
XGBoost	0.592	0.312	−0.211	0.672
OLS	0.530	0.279	−0.146	0.871

a graph-level shift cannot propagate. Bias falls to −0.044 and the slope moves to 1.575.

Neither is calibrated, and they fail in opposite directions. LayerNorm overcorrects, compressing predictions toward their mean so that they span less of the range than the truth. That is the less damaging failure: a compressed prediction is conservative, while an amplified one manufactures extremes that are not in the data. It is also the more correctable one, since a slope of 1.575 can be rescaled after the fact, whereas the unnormalized model's errors are not a simple scaling.

This result appears here and nowhere else. Across the 39 held-out states, LayerNorm was a coin flip: better R^2 in 21 of 39, better bias in 20, and worse MAE in 30. The distinction is what the two exercises ask. A held-out state is a new graph drawn from the same national covariate distribution. Ames is a new graph and a new distribution. Normalization matters for the second and not the first, which is a sharper statement of when it is needed than an average over states would give.

Spatial lag features transfer well. XGBoost with lags reaches 0.548 against LayerNorm's 0.605 and is the best-calibrated method in the comparison, with a slope of 0.910 and a bias of +0.050. It is the only method whose predictions arrive in Ames at close to the right scale. Explicit lags recover roughly nine tenths of the graph-based gain here. Together with the King County result and against the held-out states, this is the strongest support in the paper for the view that carefully engineered spatial features are a serious competitor to learned aggregation.

No method transfers without loss. Every R^2 falls by 0.2 or more from King County to Ames. Inductive architecture makes prediction on a new graph possible; it does not make the target market comparable to the source.

6.4 | Summary of empirical findings

GraphSAGE is the best or equal-best method in all three exercises. That is the consistent part, and it is stronger evidence than

the simulations could supply, because these processes were not written by us.

Two things are not consistent, and both matter more than the headline.

Spatial lag features are worth anywhere from everything to nothing. They add 0.167 of R^2 within King County and recover nine tenths of the graph-based gain transferring to Ames, but are a coin flip across 39 held-out states. Any claim that engineered lags substitute for a GNN, or that they do not, is a claim about a particular application. We can say when they helped in ours; we cannot yet say why they failed on the counties.

LayerNorm is either decisive or irrelevant. It rescues the Ames transfer, turning 0.424 into 0.605, and is a coin flip on held-out states. The separating condition appears to be whether the target region's covariate distribution differs from the source, not whether its graph is new.

Both patterns point the same way. The architecture is not what varies across these exercises; what varies is the relationship between the training region and the target region. That relationship, not the model class, is what an applied spatial prediction study needs to characterize first.

7 | Discussion and conclusion

GNNs improve spatial prediction when the process contains a neighborhood term and the evaluation is constructed so that the term survives to prediction time. Neither condition is automatic, and the second is usually left implicit.

When a GNN is worth it. Only Scenario E contains a spillover, and only there does GraphSAGE clear every alternative on a margin attributable to message passing. In the other simulated scenarios its gains come from nonlinear function approximation, which XGBoost also supplies, or fail to appear at all. The empirical exercises are more favorable, with GraphSAGE best or equal-best in all three, but the mechanism there is unverifiable in principle, since the true process is unknown.

When spatially augmented XGBoost is enough. Within King County and transferring to Ames, explicit first-order lags recover most of the graph-based gain. Across held-out states they recover none. The honest summary is that lag features are a strong and much more interpretable baseline whose adequacy is application-specific, and that a study which reports only one application cannot settle the question.

When the evaluation decides the answer. Scenario E scores 0.433 under blocked holdout and 0.240 under random holdout, for the same method, the same data, and the same seed. Random holdout builds the held-out graph at a fraction of the true density, so the lag operator at prediction time is not the operator in the data-generating equation. Under an inductive protocol, random cross-validation does not flatter graph-based methods. It handicaps them. This reverses a common expectation and, unlike most of our findings, it is a statement about protocol that transfers to any study of this kind.

Dependence and heterogeneity behave differently. Spatial dependence in the residuals, the classical spatial-statistical concern, turns out to be no resource at all for prediction into a genuinely disjoint region: no edge crosses the boundary, the spatial error model reduces to OLS, and the flexible methods overfit a field that does not transfer. Spatial heterogeneity is the live problem. It appears as the failure that LayerNorm corrects on Ames, as the level errors of up to 29 percentage points that OLS and SEM make on held-out states, and as the regime scenario we had to remove because a shift invisible in the covariates is invisible to every method. The literature's emphasis on dependence over heterogeneity is, for prediction into new regions, the wrong way round.